

A Log Buffer-Based Flash Translation Layer Using Fully-Associative Sector Translation

발표자 서혜원

발표날짜 2026.07.16

이 논문에서 해결하고자 하는 것

**“Overwrite마다 발생하는 비싼 erase 연산을
어떻게 줄일 것인가”**

INDEX

Intro	01
Background 2.1	02
Background 2.2	03
저번 발표 피드백	04

Intro

Flash memory의 문제

- overwrite 발생 시 erase가 필수적
 - 연산속도: **read < write < erase**
 - **write/read: sector단위 / erase: block단위**
- bottleneck의 원인이 됨

기존 해결책

- Overwrite으로 인해 발생하는 erase연산 줄이기 → FTL 계층 도입
 - FTL이 매핑 테이블을 이용하여 write 요청마다 즉시 erase를 하지 않기 위해
- 여러 FTL 기법 중 **Log Block Scheme(=BAST)**가 최고성능을 가짐
 - log block이 overwrite를 임시로 받는 버퍼역할 → erase횟수 감소

Intro

기존 기법인 BAST의 한계

- 기존 Log Block Scheme == BAST(Block-Associative Sector Translation)
- BAST는 overwrite를 자신에게 **전용** **배정된 log block**으로만 보낼 수 있음
- 이러한 방식이 **log block의 낮은 공간 활용률** 초래하는 근본 원인이라 주장

=> 이 논문이 제안하는 FAST 기법

- Overwrite된 sector를 아무 로그블록에나 배치 가능

2.1 Flash Memory and the Flash Translation Layer

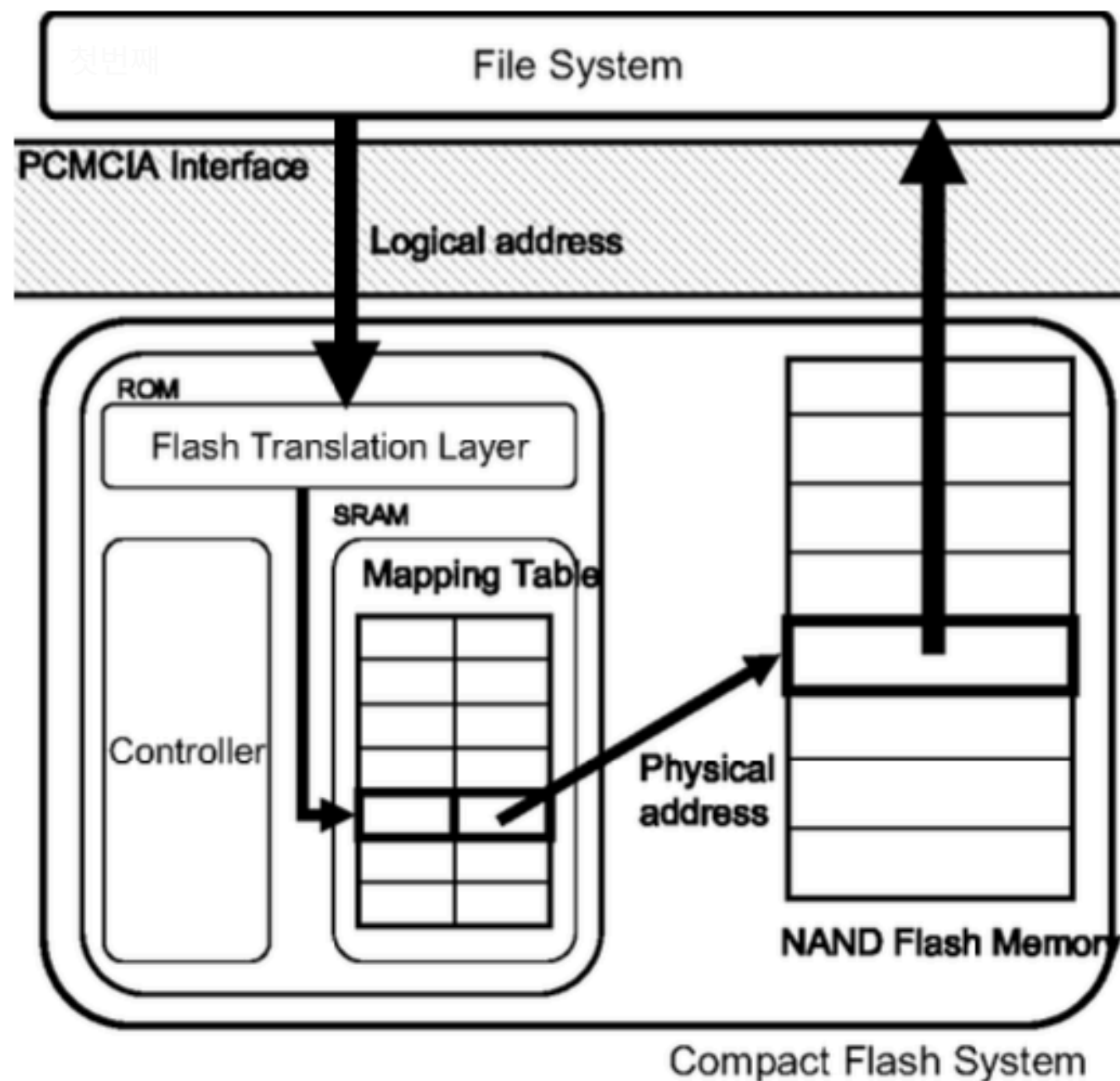


Fig. 1. An anatomy of the NAND flash memory system.

<NAND형 flash memory 구성>

- Controller
 - ROM
 - FTL
- SRAM
- PCMCIA 호스트 인터페이스
- NAND Flash Memory

2.1 Flash Memory and the Flash Translation Layer

첫번째

<FTL의 역할>

- **logical sector → physical sector 주소변환**
- 데이터 일관성 보장 및 uniform wear leveling

2.1 Flash Memory and the Flash Translation Layer

<FTL의 주소 매핑 문제>

- sector-level address mapping
 - 논리섹터 - flash memory내의 임의의 물리섹터 <매우 유연>
 - cons) Mapping table의 크기가 너무 커짐
 - e.g., flash memory size=1GB, sector size=512bytes, mapping entry=4bytes
일 경우 → mapping table size=4bytes*2M개=8MB
- block-level address mapping
 - 논리 섹터 주소 = block num + offset
 - 매핑정보는 작아짐
 - cons) 논리섹터에 대해 overwrite 발생 시, 해당 블록이 빈 물리 블록으로 remapping → bottleneck 발생

2.1 Flash Memory and the Flash Translation Layer

첫번째

**두 극단적인 매핑방식의 단점을 극복하기 위해,
BAST는 하이브리드 방식 채택**

2.2 The BAST Scheme: An Overview

- Flash memory에 대한 쓰기 함수: `write(lsn, data)`

<FTL이 물리적 위치를 찾는 법>

1. $lbn = lsn // (\text{block당 sector 개수})$; 몫 계산
2. block-level mapping table에서 해당 lbn에 상응하는 pbn 찾기
3. $offset = lsn \% (\text{block당 sector 개수})$; 나머지 계산
4. data block(=physical block)내의 해당 offset에 sector data 기록



If 이미 기록된 data block내의 sector에 overwrite 발생한다면?

2.2 The BAST Scheme: An Overview

<BAST 기법이 생기기 전>



Overwrite 발생 시,

1. 주어진 sector data를 free-block list에서 할당된 free-block의 동일한 offset에 기록
2. data block에 기록된 나머지 모든 섹터를 해당 free-block으로 복사
3. 예전 data를 가지고 있는 block 전체를 erase하고, 해당 block을 free-block list에 반환

→ Overwrite이 발생할 때마다 다수의 sector 복사 및 삭제 작업이 매번 발생!

=Merge operation 발생

2.2 <The BAST Scheme> Example

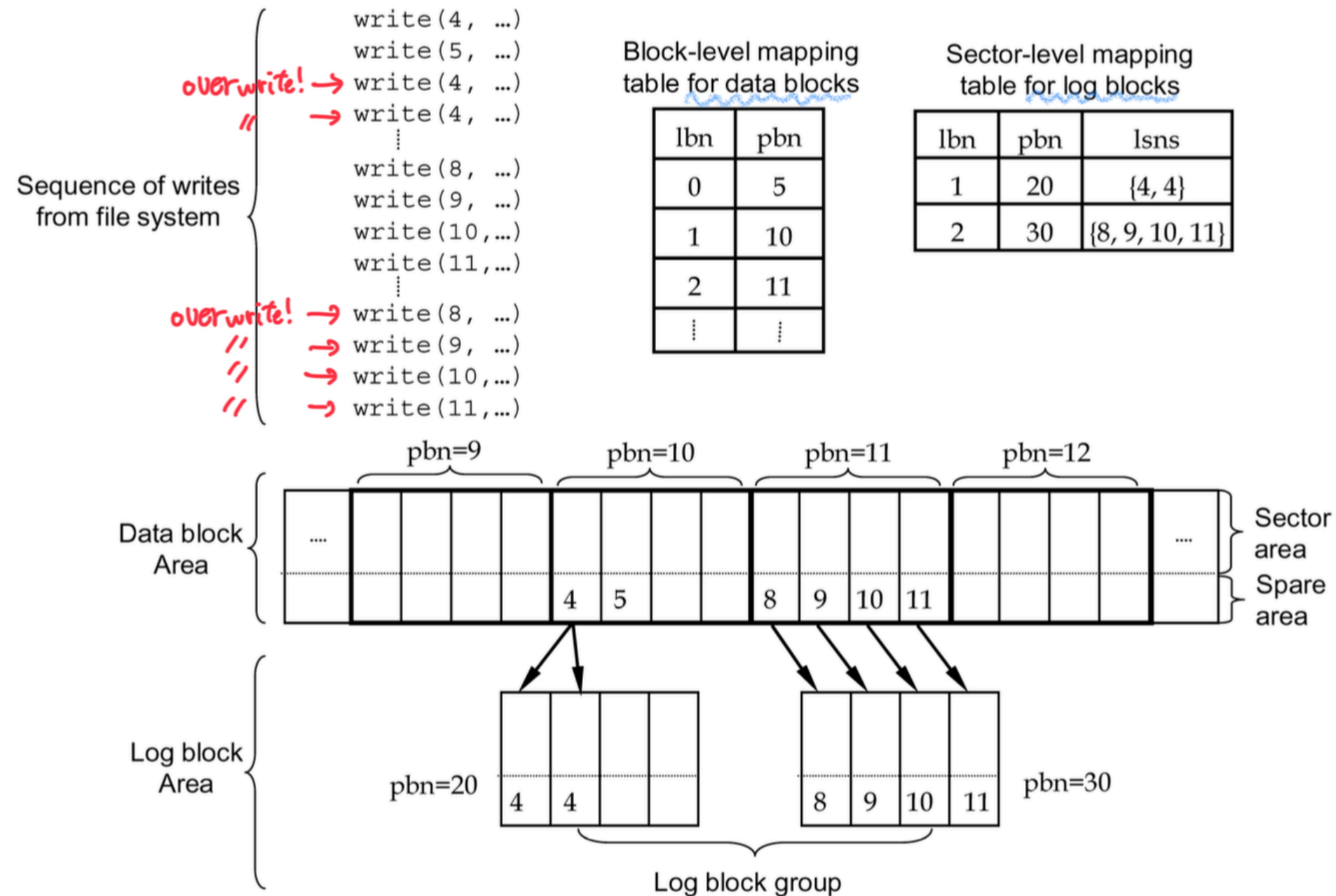


Fig. 2. Processing write operations in the BAST scheme.

2.2 The BAST Scheme Example

 fig2에 명시된 요청이후로 계속 write(4,...)가 들어온다면?

→ log block이 꽉 찰 때까지 기록

→ 빈 자리가 없을 시, merge발생 (최신값만 살림)

 data block 12에서 overwrite이 발생했는데, log block은 이미 다 사용중임.

→ 사용중인 log block 중 victim log block을 선택!

<victim log block 선택방법>

1. 조건을 따져 switch할 수 있는지 확인
2. switch가 불가하면 일반 merge로 처리

2.2 The BAST Scheme Example

Switch가 가능하기 위한 조건

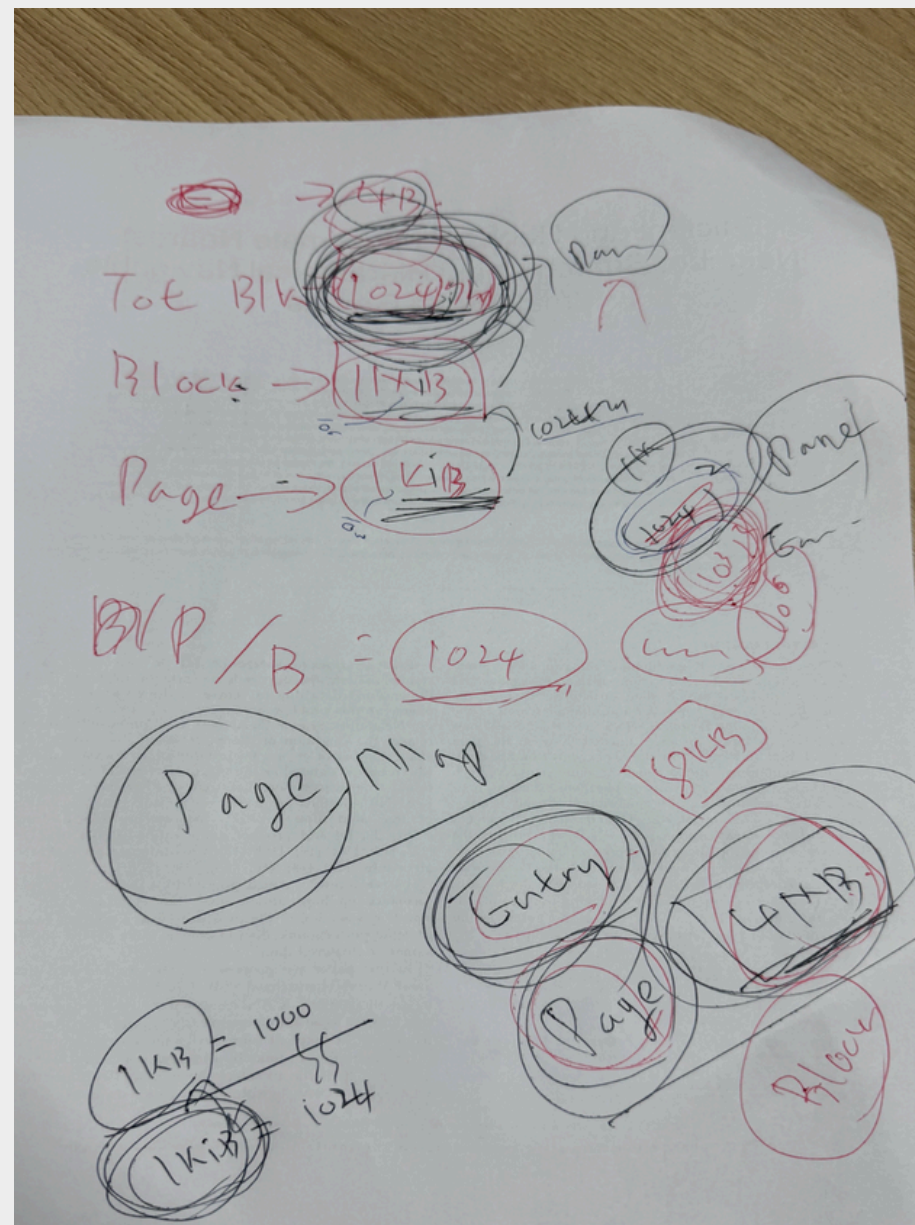
1. 첫 offset에 있는 lsn이 block 크기의 배수여야 함
2. log block이 처음부터 끝까지 순차적으로, block크기만큼 짝 차 있어야 함

Switch가 Merge보다 연산량이 작은 이유

- Switch
 - 포인터를 바꿈으로써 매핑정보 교체
 - 옛날 데이터 블록 erase연산 1번
- Merge
 - log block에서 sector read해서 free block에 write
 - 안 건드린 sector read해서 free block에 write
 - 원래 데이터 블록+로그블록 erase 연산 2번

저번 발표 피드백

- Nand flash는 write보다 erase가 느리기 때문에 사용자명령 delete 존재X
- Mapping table 크기 구하기 수식 계산



Page entry 4Byte
 Tot Block : 1024MB
 Block → 1MB
 Page → 1KB
 (1 Block = 1024 page)

$$\therefore \text{Mapping table 크기} = 4\text{Byte} \times 1024$$

$$= 4 \times \frac{1024 \times 1024}{1024}$$

$$= 4\text{KB}$$

* Mapping table 크기 8KB로 되려면

기판에 8KB, Block 크기 2MB
 기판에 8KB, Page 크기 4KB

THANK YOU